Análise Programação de Socket UDP e TCP

#LAB3

Repositório no Github: https://github.com/Maia20-21/LAB3-Sockets/

Nome	RA
Bruna Amorim Maia	10431883
Livia Negrucci Cantowitz	10389419
Rute Willemann	10436781

Questão 1: ANÁLISE

a) Execute o cliente TCP antes de executar o servidor TCP. O que acontece? Por quê?

R: Ao executar o cliente TCP antes da inicialização do servidor, a conexão não é estabelecida, resultando em erro. Isso ocorre porque o cliente tenta se conectar a um serviço ainda indisponível, uma vez que o servidor não está em execução. Como o protocolo TCP requer o estabelecimento prévio de uma conexão para a troca de dados, a comunicação não pode ser realizada nesse cenário.

```
PS C:\Users\maiab\OneDrive\Documentos\Faculdade\Matérias\RC\Lab3\TCP_python> python ClientTCP.py
Digite sua mensagem para o servidor: oi, tudo bem?
Traceback (most recent call last):
  File "C:\Users\maiab\OneDrive\Documentos\Faculdade\Matérias\RC\Lab3\TCP_python\ClientTCP.py", line 12, in <module>
    cliente.connect((TCP_IP, TCP_PORTA))
ConnectionRefusedError: [WinError 10061] Nenhuma conexão pôde ser feita porque a máquina de destino as recusou ativamente
PS C:\Users\maiab\OneDrive\Documentos\Faculdade\Matérias\RC\Lab3\TCP_python>
```

Figura 1 – Erro de conexão ao executar o cliente TCP antes da inicialização do servidor.

b) Faça o mesmo procedimento para o cliente e servidor UDP. O resultado foi similar ao socket TCP? Compare os resultados e justifique.

R: No caso do UDP, ao executar o cliente antes do servidor, não há ocorrência de erro. O cliente envia a mensagem normalmente, mesmo na ausência de um servidor ativo. Isso se deve ao fato de que o protocolo UDP é não orientado à conexão e não realiza o processo de handshaking, ou seja, não há estabelecimento prévio de conexão entre as partes. Dessa forma, os dados são simplesmente enviados ao endereço e à porta especificados, sem qualquer verificação da disponibilidade de um receptor.

```
PS C:\Users\maiab\OneDrive\Documentos\Faculdade\Matérias\RC\Lab3\UDP_python> python ClientUDP.py
Endereço IP de destino: 127.0.0.1
Porta UDP de destino: 5005
Mensagem enviada: Hello, World!
PS C:\Users\maiab\OneDrive\Documentos\Faculdade\Matérias\RC\Lab3\UDP_python>
```

Figura 2 – Execução do cliente UDP sem servidor ativo, com envio da mensagem sem ocorrência de erro.

c) O que acontece se o número da porta que o cliente tentar se conectar for diferente da porta disponibilizada pelo servidor?

R: Quando o cliente utiliza uma porta diferente da porta em que o servidor está escutando, a comunicação não é estabelecida. No TCP, isso normalmente resulta em erro de conexão (como timeout ou conexão recusada), pois não há nenhum serviço ativo naquela porta para responder à solicitação.

```
PS C:\Users\maiab\OneDrive\Documentos\Faculdade\Matérias\RC\Lab3\TCP_python> python ClientTCP.py
Digite sua mensagem para o servidor: teste
Traceback (most recent call last):
  File "C:\Users\maiab\OneDrive\Documentos\Faculdade\Matérias\RC\Lab3\TCP_python\ClientTCP.py", line 12, in <module>
    cliente.connect((TCP_IP, TCP_PORTA))
ConnectionRefusedError: [WinError 10061] Nenhuma conexão pôde ser feita porque a máquina de destino as recusou ativamente
PS C:\Users\maiab\OneDrive\Documentos\Faculdade\Matérias\RC\Lab3\TCP_python>
```

Figura 3 – Erro de conexão ao utilizar porta diferente da configurada no servidor TCP.

Questão 2 - CHAT CLIENTE-SERVIDOR (TCP)

Foi implementado um chat utilizando sockets TCP (Transmission Control Protocol), permitindo a comunicação contínua entre cliente e servidor.

Inicialmente, o servidor é iniciado e permanece em estado de escuta, aguardando conexões em uma porta definida. Em seguida, o cliente é executado e estabelece a conexão com o servidor.

Após a conexão ser estabelecida, ocorre a troca de mensagens de forma bidirecional entre as duas partes. A comunicação permanece ativa até que uma das extremidades envie o comando QUIT, encerrando a conexão.

Por utilizar o protocolo TCP, a aplicação garante a entrega dos dados e mantém o controle da conexão durante toda a comunicação.

```
● @Maia20-21 ➜/workspaces/LAB3-Sockets (main) $ cd q2-chat/tcp
● @Maia20-21 ➜/workspaces/LAB3-Sockets/q2-chat/tcp (main) $ ls
  client.py  server.py
⊙ @Maia20-21 ➜/workspaces/LAB3-Sockets/q2-chat/tcp (main) $ python server.py
  Traceback (most recent call last):
    File "/workspaces/LAB3-Sockets/q2-chat/tcp/server.py", line 7, in <module>
      server.bind((HOST, PORT))
  OverflowError: bind(): port must be 0-65535.
● @Maia20-21 ➜/workspaces/LAB3-Sockets/q2-chat/tcp (main) $ python server.py
  Servidor aguardando conexão...
  Conectado por ('127.0.0.1', 38220)
  Cliente: oi
  Servidor: tudo bem?
  Cliente: eu estou OTIMA
  Servidor: que legallll
  Cliente: tem alguma novidade por ai?
  Servidor: nao tantas... a area continua bem parada, nao temos as metas definidas ainda.
  Cliente: hmm que pena, mas quando definirem as metas, a bomba vai chegar!
  Servidor: SIM!
  Cliente encerrou
```

Figura 4 – Execução do servidor TCP com conexão estabelecida e troca de mensagens com o cliente.

```
● @Maia20-21 ➜/workspaces/LAB3-Sockets (main) $ cd q2-chat/tcp
● @Maia20-21 ➜/workspaces/LAB3-Sockets/q2-chat/tcp (main) $ python client.py
  Cliente: oi
  Servidor: tudo bem?
  Cliente: eu estou OTIMA
  Servidor: que legallll
  Cliente: tem alguma novidade por ai?
  Servidor: nao tantas... a area continua bem parada, nao temos as metas definidas ainda.
  Cliente: hmm que pena, mas quando definirem as metas, a bomba vai chegar!
  Servidor: SIM!
  Cliente: QUIT
○ @Maia20-21 ➜/workspaces/LAB3-Sockets/q2-chat/tcp (main) $ 
```

Figura 5 – Execução do cliente TCP realizando comunicação bidirecional e encerramento com comando QUIT.

Questão 2 - COMUNICAÇÃO UDP

A implementação utilizando sockets do o protocolo UDP (User Datagram Protocol) estabelece uma comunicação não orientada à conexão. Isso significa que não há uma negociação de início de conversa entre servidor e cliente; o servidor simplesmente monitora a porta e processa as mensagens recebidas. Diferentemente do TCP, o UDP não oferece garantias de entrega ou controle de ordem das mensagens, resultando em uma comunicação mais rápida, mas menos rigorosa.

Nos testes realizados, observou-se esta característica: o cliente consegue enviar dados mesmo com o servidor inativo, sem gerar erros imediatos de sistema. No entanto, devido à natureza do UDP, se o servidor estiver inativo, a mensagem é enviada para o "vazio" da rede e o cliente, caso esteja esperando uma resposta, ficará bloqueado indefinidamente aguardando uma resposta que nunca chegará, pois não há um mecanismo de timeout ou de verificação de status da porta remota implementado. O encerramento do chat é manual, em ambas as pontas, utilizando o comando QUIT, que finaliza a execução do programa.

```
@rutewillemann → /workspaces/LAB3-Sockets/q2-chat/udp (main) $ python server.py
Servidor UDP aguardando mensagens em 10436...
```

Figura 6 - Inicialização do Servidor UDP e entrando em estado de escuta passiva.

```
@rutewillemann → /workspaces/LAB3-Sockets/q2-chat/udp (main) $ python client.py
Cliente: Ola, servidor! Recebeu minha mensagem?
```

Figura 7 - Cliente enviando datagramas de texto para o IP e porta destino do servidor.

```
@rutewillemann → /workspaces/LAB3-Sockets/q2-chat/udp (main) $ python server.py
Servidor UDP aguardando mensagens em 10436...
Cliente disse: Ola, servidor! Recebeu minha mensagem?
Servidor: Sim cliente, recebi sua mensagem :)
O cliente encerrou o chat.
@rutewillemann → /workspaces/LAB3-Sockets/q2-chat/udp (main) $
```

Servidor processa mensagem e identifica endereço de origem do cliente para resposta.

```
@rutewillemann → /workspaces/LAB3-Sockets/q2-chat/udp (main) $ python server.py
Servidor UDP aguardando mensagens em 10436...
Cliente disse: Ola, servidor! Recebeu minha mensagem?
Servidor: Sim cliente, recebi sua mensagem :)
O cliente encerrou o chat.
@rutewillemann → /workspaces/LAB3-Sockets/q2-chat/udp (main) $
```

```
@rutewillemann → /workspaces/LAB3-Sockets/q2-chat/udp (main) $ python client.py
Cliente: Ola, servidor! Recebeu minha mensagem?
Servidor disse: Sim cliente, recebi sua mensagem :)
Cliente: QUIT
Você encerrou o chat.
@rutewillemann → /workspaces/LAB3-Sockets/q2-chat/udp (main) $
```

Figura 9 - Fluxo de mensagens bidirecional e encerramento de sessão via comando QUIT.

Questão 3 - APLICAÇÃO

A aplicação desenvolvida é um sistema de chat com transferência de arquivos utilizando socket TCP em Python. O servidor aguarda a conexão de um cliente e, após conectado, oferece um menu de comandos. O usuário pode listar os arquivos disponíveis no servidor, baixar um arquivo do servidor para o cliente, enviar um arquivo do cliente para o servidor, e trocar mensagens de chat em tempo real. A comunicação é feita via protocolo TCP na porta 10389, garantindo a entrega confiável dos dados.

```
@LiviaCantowitz → /workspaces/LAB3-Sockets (main) $ cd "/workspaces/LAB3-Sockets/q3- Aplicacao"
python q3_servidor.py
[SERVIDOR] Rodando em localhost:10389
[SERVIDOR] Pasta de arquivos: 'arquivos_servidor/'
[SERVIDOR] Aguardando cliente...

[SERVIDOR] Cliente conectado: ('127.0.0.1', 40374)

[Cliente]: menu
```

Figura 10 - Terminal do servidor

```
@LiviaCantowitz → /workspaces/LAB3-Sockets (main) $ cd "/workspaces/LAB3-Sockets/q3- Aplicacao"
python q3_cliente.py
[CLIENTE] Conectado ao servidor localhost:10389

Servidor: Conectado! Digite MENU para ver os comandos disponíveis.

>>> menu
Servidor:

=== MENU ===
LISTAR          - Lista arquivos disponíveis no servidor
BAIXAR <arq>    - Baixa um arquivo do servidor
ENVIAR <arq>    - Envia um arquivo ao servidor
CHAT <mensagem> - Envia mensagem de chat
QUIT            - Encerra a conexão


>>>
```

Figura 11 - Terminal do cliente

```
>>> listar
Servidor:
Arquivos no servidor:
  - recebido_teste.txt
  - exemplo.txt

>>> baixar exemplo.txt
  [+] Arquivo 'exemplo.txt' salvo em 'downloads/'
>>> enviar teste.txt
  [+] Arquivo 'teste.txt' recebido com sucesso!
>>> chat ola servidor
Servidor (chat): ola cliente
>>> quit
[CLIENTE] Conexão encerrada.
@LiviaCantowitz → /workspaces/LAB3-Sockets/q3- Aplicacao (main) $
```

Figura 12 - Teste do cliente

```
[Cliente]: menu
[Cliente]: listar
[Cliente]: baixar exemplo.txt
  [+] Arquivo 'exemplo.txt' enviado ao cliente.
[Cliente]: enviar teste.txt
  [+] Arquivo salvo como 'recebido_teste.txt'.
[Cliente]: chat ola servidor
  [CHAT do cliente]: ola servidor
  Sua resposta (servidor): ola cliente
[Cliente]: quit
[SERVIDOR] Encerrando...
[SERVIDOR] Encerrado.
○ @LiviaCantowitz → /workspaces/LAB3-Sockets/q3- Aplicacao (main) $
```

Figura 13 - Teste do servidor

CONCLUSÃO

A atividade prática proporcionou uma compreensão aprofundada da programação de sockets e do comportamento dos protocolos da camada de transporte, enfatizando as distinções cruciais entre TCP e UDP.

Diferenças Chave entre TCP e UDP:

- **TCP (Transmission Control Protocol):** É orientado à conexão, exigindo um *three-way handshake* inicial. Garante a entrega confiável dos dados, sendo ideal para aplicações intolerantes à perda de informação, como o sistema de transferência de arquivos (Questão 3).

- **UDP (User Datagram Protocol):** Opera de forma não orientada à conexão, priorizando a velocidade. É adequado para comunicações simples e diretas, como o chat (Questão 2), mas requer que a aplicação trate bloqueios caso o receptor esteja inativo, pois não oferece garantias de integridade ou estado.

Desafios Encontrados:

A sincronização de envio e recebimento foi difícil devido à ausência de controle de estado em protocolos sem conexão, resultando em potencial espera indefinida. Além disso, a implementação de uma lógica de encerramento mútua exigiu profundo conhecimento do tratamento das mensagens de controle pela aplicação para assegurar a liberação correta dos recursos e evitar processos órfãos ou conflitos.

Conclusão e Importância:

Reforçou-se a relevância da comunicação cliente-servidor como base das redes modernas. O desenvolvimento da capacidade de criar aplicações que trocam informações de maneira bidirecional e confiável é uma competência essencial para a construção de sistemas distribuídos robustos, adaptáveis às diversas necessidades dos serviços de rede.